HOME WORK 3 -SOLID

قسمت اول : **SOLID**

OCP-OPEN /CLOSED PRINCIPLE

این اصل در SOLID می گوید که کلاس ها و ماژول ها برای توسعه Extensionباز هستند و برای تغییر باید بسته Modification باشند . یعنی وقتی بخواهیم قابلیت جدیدی اضافه کنیم نباید کد های قبلی را تغییر دهیم. بلکه با ایجاد کلاس جدید ، ارث بری یا Interfaceقابلیت جدید را اضافه کنیم .

توضیح نقض اصل OCP

در این کد برای محاسبه مساحت از شرط Ifاستفاده شده است .اگر بخواهیم شکل جدیدی برای محاسبه مساحت اضافه کنیم ناچار به تغییر کد های متد CalculateArea() در کلاس هستیم و شرط جدیدی به شرط های قبلی اضافه کنیم در نتیجه برای افزودن قابلیت جدید ،کدهای قبلی تغییر میکنند و این موضوع اصل OCPرا نقض میکند.

```
class Shape
{

    public double Height { get; set; }
    public double Width {  get;set; }
    public double Radius {  get;set; }
    public string Type {  get; set; }
    public double CalculateArea()
    {

        if (Type == "Rectangle")
        {
            return Height * Width;
        }
        else if (Type == "Circle")
        {
            return Radius * Radius * Math.PI;

        }
        else
        {
            return 0;

                }

    }
}
```

اصلاح کد و رعایت اصل OCP

در این طراحی هر شکل کلاس مخصوص به خود را دارد و Interface IShapeرا پیاده سازی میکند و اگر بخواهیم شکل جدیدی اضافه کنیم فقط کافیست کلاس جدیدی اضافه کنیم و Interface IShapeرا پیاده سازی کند در این حالت نیازی به تغییر کدهای قبلی نیستیم و اصل OCPرعایت شده است.

```
interface IShape
 {


     double CalculateArea();
 }
 class Rectangle : IShape
 {
     public double Width { get; set; }
      public double Height {  get; set; }
     public double CalculateArea()
     {

         return  Width * Height;

     }
 }
  class Circle:IShape

 {
     public double Radius {  get; set; }
     public double CalculateArea()
     {
         return  Radius * Radius * Math.PI;
     }
}
```

اصلSRP (Single Responsibility Principle)

بیان می‌کند که هر کلاس باید فقط یک مسئولیت یا یک دلیل برای تغییر داشته باشد. اگر یک کلاس چندین وظیفه مختلف را انجام دهد، این اصل نقض می‌شود.

در این مثال، کلاس Student علاوه بر نگهداری اطلاعات دانشجو، وظیفه نمایش اطلاعات، محاسبه معدل و ذخیره اطلاعات در فایل را نیز بر عهده دارد. بنابراین این کلاس بیش از یک مسئولیت دارد و اصل SRPرا نقض می‌کند.

نقض اصل SRP :

```
 public class Student
{

    public int StudentID { get; set; }
    public string Name { get; set; }
    public int Age { get; set; }
    public string NationalId { get; set; }
    public double Ave { get; set; }
    public double Grade1 { get; set; }
    public double Grade2 { get; set; }

    public void DisplayInfo()
    {
        Console.WriteLine($" {Name} , {Age} , {NationalId} , {StudentID}  ");
    }
    public double Average()
    {
        return Ave = (Grade1 + Grade2) / 2;
    }
    public void SaveToFile()
    {
        Console.WriteLine("Student information saved to file.");
    }
}
```

کد اصلاح شده و رعایت اصل SRP

```
namespace SRP
{

    class StudentAverage
    {
        public double Grade1 { get; set; }
        public double Grade2 { get; set; }
        public double Grade3 { get; set; }

        public double Average()
        {

            return (Grade1 + Grade2 + Grade3) / 3;
        }
    }
    class StudentRepository
    {
        public void SaveToFile()
        {
            Console.WriteLine("Student information saved to file.");
        }
    }

    class DisplayInformation
    {
```

```
        public void StudentDisplay(Student student)
        {
            Console.WriteLine($"Name: {student.Name}");
            Console.WriteLine($"Age: {student.Age}");
            Console.WriteLine($"National ID: {student.NationalId}");
            Console.WriteLine($"Student ID: {student.StudentID}");
        }
    }

}
```

اصل Liskov Substitution Principle : LSP

اصل **LSP** می گوید که هرکلاس فرزند بدون مشکل بتوانند جایگزین کلاس والد خود شود .یعنی کلاس فرزند نباید فرزندی داشته باشد که با کلاس پایه ناسازگار باشد .

نقض اصل LSP :

در این مثال، کلاس **Penguin** از کلاس **Bird** ارث‌بری کرده است، اما پنگوئن توانایی پرواز ندارد. با این حال، به دلیل وجود متد **()Fly** در کلاس **Bird**، کلاس **Penguin** نیز مجبور به پیاده‌سازی این متد شده است. بنابراین کلاس **Penguin** نمی‌تواند به‌درستی جایگزین کلاس **Bird** شود و اصل **Liskov Substitution Principle (LSP)** نقض می‌شود.

```
class Bird
{

    public virtual void Fly()
    {
        Console.WriteLine("Bird is Flying");
    }

}

class Sparrow : Bird
{


    public override void Fly()
    {
        Console.WriteLine("Sparrow is Flying");
    }

}
class Penguin : Bird
```

```
{
    public override void Fly()
    {
        Console.WriteLine("Penguin is Flying");
    }

}
```

اصلاح کد ورعایت اصل LSP :

```
interface IFlyable
{
    void Fly();
}

class Bird {

    public string Name { get; set; }
    public int Age {  get; set; }
}

class Sparrow : Bird, IFlyable {
    public void Fly() {
        Console.WriteLine("Sparrow is flying");
        }
}
class Penguin:Bird
{
}
```

توضیح رعایت اصل LSP :

در این طراحی، کلاس `Bird` فقط ویژگی‌های مشترک همه پرندگان را نگهداری می‌کند. قابلیت پرواز در اینترفیس `IFlyable` قرار گرفته است و فقط پرندگانی که توانایی پرواز دارند، مانند `Sparrow`، اینترفیس `IFlyable` را پیاده‌سازی می‌کنند. کلاس `Penguin` فقط از `Bird` ارث‌بری می‌کند و متد `()Fly` ندارد، زیرا توانایی پرواز ندارد. بنابراین هر کلاس فرزند رفتار متناسب با خود را دارد و می‌تواند بدون ایجاد مشکل جایگزین کلاس پایه شود؛ در نتیجه اصل `LSP` رعایت شده است.

اصل ISP :

اصل Interface Segregation Principle (ISP) می‌گوید که هیچ کلاسی نباید مجبور شود متدهایی را پیاده‌سازی کند که به آن‌ها نیازی ندارد. به جای استفاده از یک Interface بزرگ، بهتر است چند Interface کوچک و تخصصی ایجاد شود تا هر کلاس فقط متدهای موردنیاز خود را پیاده‌سازی کند.

مثال نقض ISP :

```
using System.Runtime.InteropServices;

interface IMachine
{

    void Print();
    void Scan();

}
class Scanner:IMachine
{
public void Print()
    {
        Console.WriteLine("Printer is printing.");


    }
    public void Scan() {
        Console.WriteLine("Printer cannot scan.");

    }
}
class Printer : IMachine
{
    public void Print()
    {
        Console.WriteLine("Printer is printing.");

    }
    public void Scan()
    {
        Console.WriteLine("Scanner is scanning.");

    }

}
```

در این مثال، Interface IMachine شامل دو متد Print() و Scan() است. کلاس Printer فقط قابلیت چاپ دارد و کلاس Scanner فقط قابلیت اسکن دارد، اما هر دو کلاس مجبور شده‌اند هر دو متد را پیاده‌سازی کنند. این موضوع باعث می‌شود کلاس‌ها به متدهایی وابسته باشند که به آن‌ها نیازی ندارند و در نتیجه اصل Interface Segregation Principle (ISP)نقض می‌شود.

رعایت اصل ISP و اصلاح کد :

برای رعایت اصل ISP، Interfaceبزرگ IMachineبه دو Interface کوچک‌تر به نام‌های IPrinterو IScannerتقسیم شده است. کلاس Printerفقط Interface IPrinter را پیاده‌سازی می‌کند و کلاس Scannerفقط Interface IScanner را پیاده‌سازی می‌کند. در نتیجه هر کلاس فقط متدهای موردنیاز خود را دارد و مجبور به پیاده‌سازی متدهای غیرضروری نیست. بنابراین اصل Interface Segregation Principle (ISP)رعایت می‌شود.

```
interface IPrinter
{
    void Print();
}
interface IScanner
{
    void Scan();
}

class Scanner:IScanner
{
    public void Scan() {
        Console.WriteLine("Scanner is scanning.");

    }
}
class Printer : IPrinter
{
    public void Print()
    {
        Console.WriteLine("Printer is printing.");

    }


}
```

اصل DIP:

اصل Dependency Inversion Principle (DIP)می‌گوید کلاس‌های سطح بالا نباید مستقیماً به کلاس‌های سطح پایین وابسته باشند، بلکه هر دو باید به یک انتزاع مانند Interfaceیا Abstract Class وابسته باشند. این کار باعث می‌شود تغییر یا توسعه برنامه بدون تغییر کلاس‌های اصلی انجام شود.

نقض اصل DIP :

```
class NotificationService

{
    public void Notify()
    {
        EmailSender sender = new EmailSender();
        sender.Send();

    }
}


class EmailSender
{
    public void Send()
    {

        Console.WriteLine("Email sent.");
    }
}

class Program
{
    public static void Main()
    {

    }


}
```

توضیح نقض اصل **DIP** :

در این مثال، کلاس **NotificationService** مستقیماً به کلاس **EmailSender** وابسته است و شیء آن را ایجاد می‌کند. اگر در آینده بخواهیم به جای ارسال ایمیل، از پیامک یا روش دیگری برای ارسال اعلان استفاده کنیم، باید کلاس **NotificationService** را نیز تغییر دهیم. بنابراین کلاس سطح بالا به کلاس سطح پایین وابسته است و اصل **Dependency Inversion Principle (DIP)** نقض می‌شود.

کد اصلاح شده و رعایت اصل DIP

```
interface INotificationSender
{
    void Send();
}

class EmailSender : INotificationSender
{
    public void Send()
    {
        Console.WriteLine("Email sent.");
    }
}

class NotificationService
{
    private INotificationSender sender;

    public NotificationService(INotificationSender sender)
    {
        this.sender = sender;
    }

    public void Notify()
    {
        sender.Send();
    }
}

class Program
{
    public static void Main()
    {
        INotificationSender sender = new EmailSender();

        NotificationService notification = new NotificationService(sender);

        notification.Notify();
    }
```

توضیح رعایت اصل DIP

در این طراحی، کلاس NotificationServiceبه جای وابستگی مستقیم به کلاس EmailSender، به Interface INotificationSenderوابسته است. به همین دلیل می‌توان بدون تغییر کلاس NotificationService، روش‌های جدیدی مانند SmsSenderیا WhatsAppSenderرا اضافه کرد. در نتیجه اصل Dependency Inversion Principle (DIP)رعایت می‌شود.
